



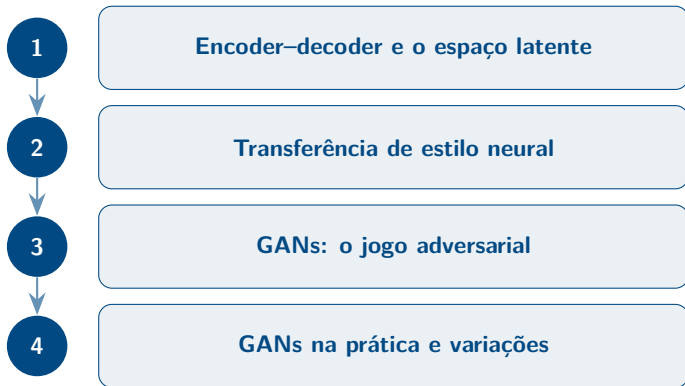
Modelos Generativos: Encoder–Decoder, Transferência de Estilo e GANs

Aprendizagem de Máquina — CDIA

Prof. Dr. Rooney R. A. Coelho

Graduação em Ciência de Dados e Inteligência Artificial — CDIA

2026



Um notebook por tema: `01_encoder_decoder`, `02_transferencia_de_estilo`, `03_gan`.

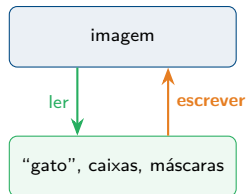
Até aqui, redes que *leem* imagens

O caminho até agora:

- CNNs: classificar (*o que* há na imagem);
- Detecção: localizar (*onde* está);
- Segmentação: delimitar pixel a pixel.

Hoje invertemos a seta: redes que **escrevem** imagens.

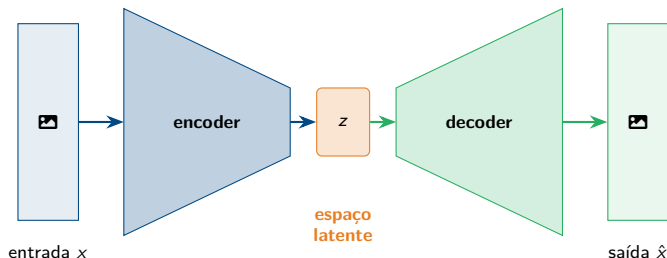
- Reconstruir (autoencoder);
- Repintar (transferência de estilo);
- Criar do zero (GANs).





Encoder–Decoder e o espaço latente

A arquitetura encoder-decoder



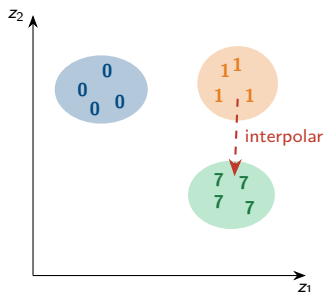
- O **encoder** comprime a entrada em um vetor pequeno z (o *gargalo*);
- o **decoder** reconstrói a entrada a partir de z ;
- a rede é obrigada a guardar em z **só o essencial**.

O espaço latente: um mapa comprimido dos dados

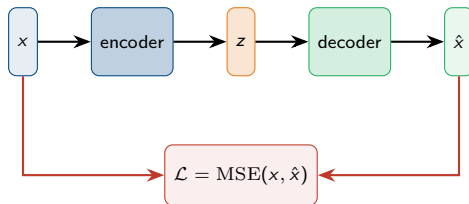
- Cada imagem vira um **ponto** no espaço latente;
- imagens parecidas $\rightarrow$ pontos **próximos**;
- a rede organiza o espaço sozinha, sem rótulos;
- andar pelo espaço latente = transformar imagens de forma contínua.

Por que “latente”?

São fatores escondidos (não observados) que explicam os dados: traço, inclinação, espessura...



Autoencoder: aprender a reconstruir



- O alvo é **a própria entrada**: aprendizado **auto-supervisionado** (sem rótulos);
- a perda mede o erro de reconstrução pixel a pixel;
- o gargalo impede a solução trivial de “copiar” a imagem.

Encoder e decoder em PyTorch

```
DIM_LATENTE = 32

encoder = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, DIM_LATENTE), # 784 -> 32: o gargalo
)

decoder = nn.Sequential(
    nn.Linear(DIM_LATENTE, 256), nn.ReLU(),
    nn.Linear(256, 784), nn.Sigmoid(), # pixels de volta a [0, 1]
    nn.Unflatten(1, (1, 28, 28)),
)

autoencoder = nn.Sequential(encoder, decoder).to(device)
```

O MNIST tem $28 \times 28 = 784$ pixels; o latente usa só 32 números ($24\times$ menor).

Treinamento: cinco linhas que importam

```
otimizador = torch.optim.Adam(autoencoder.parameters(), lr=1e-3)
mse = nn.MSELoss()

for epoca in range(5):
    for x, _ in loader: # o rótulo é descartado!
        x = x.to(device)
        perda = mse(autoencoder(x), x)
        otimizador.zero_grad(); perda.backward(); otimizador.step()
```

- Repare no `_`: os rótulos existem no MNIST, mas **não são usados**;
- o mesmo código serve para qualquer conjunto de imagens.

Reconstruções após 5 épocas

Original



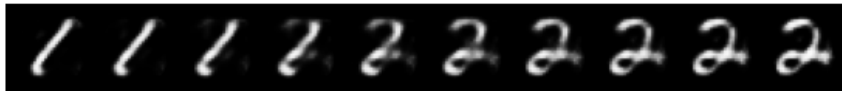
Reconstruída



A rede recupera o dígito passando por um gargalo de 32 números — pequenas imperfeições são o preço da compressão.

Interpolação: andar pelo espaço latente

Interpolação no espaço latente

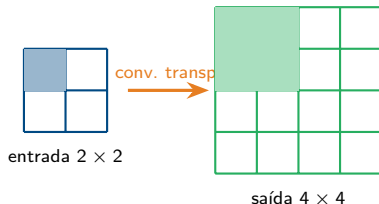


Decodificando pontos no segmento entre z_1 e z_2 : a transição é **suave**, prova de que o espaço latente tem estrutura — não é um amontoado de códigos.

Convolução transposta: upsampling aprendido

- Convolução comum **reduz** o mapa (com stride);
- a **convolução transposta** faz o inverso: **amplia**;
- cada pixel de entrada “pinta” um bloco da saída, com pesos **aprendidos**;
- é o bloco básico de decoders convolucionais (e do DCGAN, no bloco 4).

`nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1)` dobra a resolução.



Denoising: a mesma rede, um truque novo

Entrada ruidosa



Saída da rede



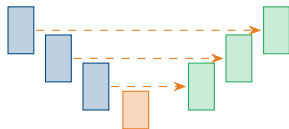
Original



Entrada **com ruído**, alvo **limpo**: para resolver a tarefa, a rede precisa aprender **como um dígito de verdade se parece** — e não decorar pixels.

Encoder-decoder está em toda parte

- **U-Net** (aula passada): encoder-decoder com *skip connections* para segmentação;
- **tradução automática**: frase $\rightarrow$ vetor $\rightarrow$ frase em outra língua;
- **compressão neural** de imagens e áudio;
- **fast style transfer**: uma rede encoder-decoder aplica o estilo em tempo real (bloco 2).



U-Net: encoder-decoder com atalhos

Por que o autoencoder não é um bom gerador?

- Tentação: jogar um z aleatório no decoder e “criar” um dígito novo;
- problema: o treino só cobre os z **vindos de imagens reais**;
- entre as regiões povoadas há **buracos** — z sorteado ao acaso costuma cair neles e o decoder devolve borrões.

O salto conceitual da aula

Para **gerar** dados novos, não basta reconstruir: é preciso aprender a **distribuição** dos dados — $p(x)$ — e amostrar dela. É exatamente isso que as **GANs** fazem (bloco 3).



01_encoder_decoder.ipynb

- 1 Montar encoder e decoder ([nn.Sequential](#) de poucas linhas);
- 2 treinar o autoencoder no MNIST **sem usar rótulos**;
- 3 comparar originais e reconstruções;
- 4 interpolar dois dígitos no espaço latente;
- 5 treinar um *denoising autoencoder*;
- 6 exercícios: gargalo de dimensão 2 e ruído extremo.

Tudo que não é o tema da aula (dados, gráficos) está em [aula_generativa.py](#).

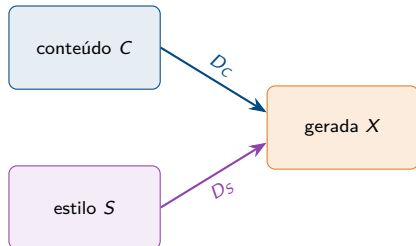


Transferência de Estilo Neural

A ideia: três imagens

Gatys, Ecker e Bethge (2015): *A Neural Algorithm of Artistic Style*.

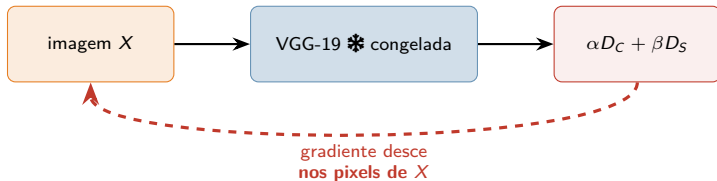
- Imagem de **conteúdo** C : o que aparece;
- imagem de **estilo** S : a textura, a pincelada;
- imagem **gerada** X : começa como cópia de C e é modificada aos poucos.



Duas distâncias

X minimiza $\alpha D_C(X, C) + \beta D_S(X, S)$

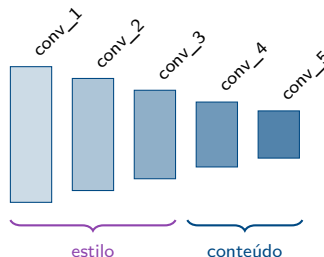
A grande inversão: otimizar a imagem, não a rede



- Nas aulas anteriores: dados fixos, **pesos** otimizados;
- aqui: **pesos fixos**, **imagem** otimizada (X vira parâmetro com `requires_grad`);
- a VGG-19 pré-treinada na ImageNet é só um **instrumento de medida**.

VGG-19: régua em profundidades diferentes

- Camadas **rasas** (`conv_1`, `conv_2`):
bordas, cores, texturas → boas para medir **estilo**;
- camadas **profundas** (`conv_4`, `conv_5`):
objetos e disposição → boas para medir **conteúdo**;
- usamos `vgg19().features` em modo `.eval()`, sem gradiente nos pesos.



Perda de conteúdo: um módulo que só observa

$$D_C = \text{MSE}(F^{(\ell)}(X), F^{(\ell)}(C)) \quad \ell = \text{conv_4}$$

```
class ContentLoss(nn.Module):
    def __init__(self, alvo):
        super().__init__()
        self.alvo = alvo.detach() # features do conteúdo, fixas

    def forward(self, x):
        self.loss = F.mse_loss(x, self.alvo)
        return x # devolve a entrada intacta
```

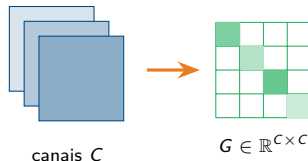
O módulo é inserido **dentro** da VGG: mede a perda de passagem e deixa o fluxo seguir.

Matriz de Gram: a assinatura do estilo

- Estilo $\neq$ posição: é **quais texturas aparecem juntas**;
- achatamos cada canal e medimos a correlação entre canais:

$$G = \frac{F F^T}{C \cdot hw} \in \mathbb{R}^{C \times C}, \quad F \in \mathbb{R}^{C \times hw}$$

- G_{ij} alto = os canais i e j disparam juntos;
- a posição espacial é **descartada** na soma.



Perda de estilo: Gram contra Gram

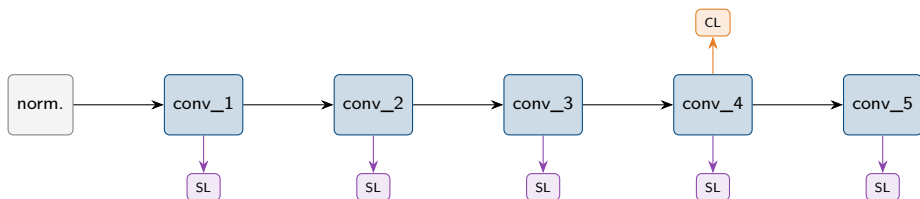
$$D_S = \sum_{\ell \in \{\text{conv_1} \dots \text{conv_5}\}} \text{MSE}(G^{(\ell)}(X), G^{(\ell)}(S))$$

```
def matriz_gram(x):
    b, c, h, w = x.size()
    feats = x.view(b * c, h * w)
    return (feats @ feats.t()) / (b * c * h * w)

class StyleLoss(nn.Module):
    def __init__(self, alvo):
        super().__init__()
        self.alvo = matriz_gram(alvo).detach()

    def forward(self, x):
        self.loss = F.mse_loss(matriz_gram(x), self.alvo)
        return x
```

Onde as perdas entram na VGG



- `StyleLoss` (SL) após `conv_1`–`conv_5`; `ContentLoss` (CL) após `conv_4`;
- a montagem dessa “VGG recheada” está pronta na biblioteca:
`ag.montar_modelo_estilo(ContentLoss, StyleLoss, conteudo, estilo);`
- tudo depois da última perda é descartado — economia de cômputo.

O loop: gradiente descendendo na imagem

```
imagem = conteudo.clone().requires_grad_(True) # X vira parâmetro!
otimizador = torch.optim.Adam([imagem], lr=0.02)
PESO_ESTILO, PESO_CONTEUDO = 1e6, 1.0

for passo in range(301):
    otimizador.zero_grad()
    modelo(imagem) # perdas se atualizam na passagem
    perda_e = sum(p.loss for p in perdas_estilo) * PESO_ESTILO
    perda_c = sum(p.loss for p in perdas_conteudo) * PESO_CONTEUDO
    (perda_e + perda_c).backward()
    otimizador.step()
    with torch.no_grad():
        imagem.clamp_(0, 1) # mantém pixels válidos
```

Resultado

Conteúdo



Estilo



Gerada



Da esquerda para a direita: conteúdo, estilo e gerada ($\text{PESO_ESTILO} = 10^6$, $\text{PESO_CONTEUDO} = 1$). O conteúdo continua reconhecível; as texturas e cores vêm da pintura. O equilíbrio é controlado por α (conteúdo) e β (estilo).

Custo e a ponte com o bloco 1

- Cada imagem exige uma **otimização completa** (centenas de passos) — segundos a minutos por foto;
- **Fast style transfer** (Johnson et al., 2016): treinar **uma rede encoder-decoder** que aplica o estilo em uma única passagem;
- a perda de Gatys vira a *função de treino* dessa rede — os filtros de apps de celular funcionam assim;
- mesma receita da aula: um conceito caro (otimização) destilado em uma rede rápida (encoder-decoder).

Prática: `02_transferencia_de_estilo.ipynb`

Carregar conteúdo e estilo → definir `ContentLoss`, `matriz_gram`, `StyleLoss` → montar a VGG recheada (1 chamada) → otimizar a imagem → exercícios com α/β e foto própria.



GANs: o jogo adversarial

Generative Adversarial Nets

Ian J. Goodfellow,¹ Jean Pouget-Abadie,¹ Mehdi Mirza,¹ Bing Xu,¹ David Warde-Farley,¹
Sherjil Ozair,² Aaron Courville,¹ Yoshua Bengio¹
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

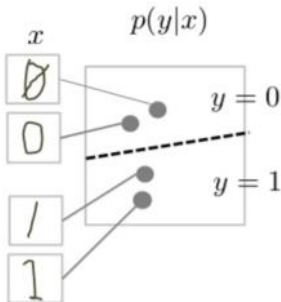
We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.



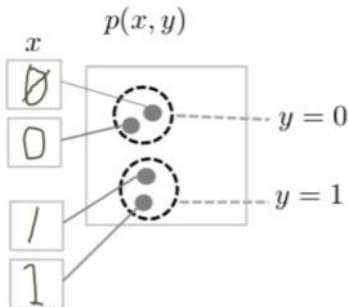
Goodfellow et al., *Generative Adversarial Nets* (NeurIPS 2014): treinar **duas redes em competição** — um gerador G e um discriminador D .

Modelos discriminativos vs. generativos

- Discriminative Model



- Generative Model



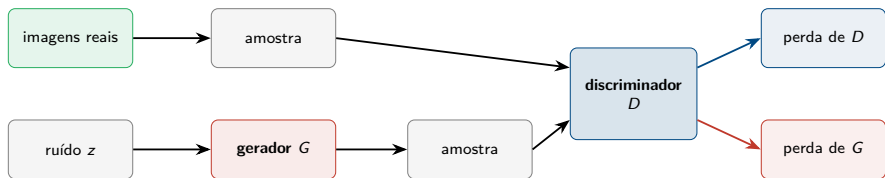
- **Discriminativo:** aprende a fronteira — $p(y | x)$ (toda a 1ª metade do curso);
- **generativo:** aprende como os dados se espalham — $p(x)$ ou $p(x, y)$ — e pode **amostrar exemplos novos**;
- GANs são modelos generativos *implícitos*: geram amostras sem escrever $p(x)$.

Nenhuma destas pessoas existe



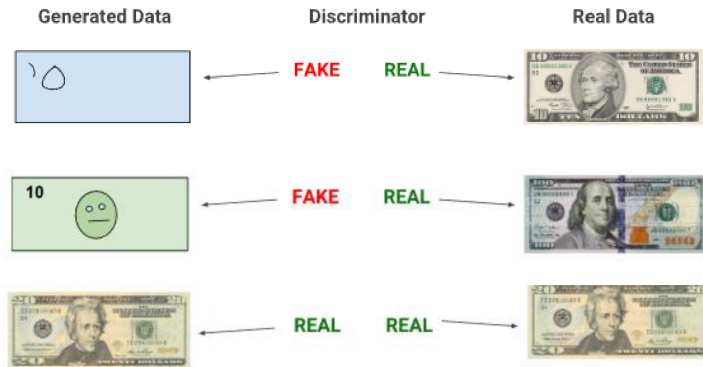
Rostos gerados por uma GAN progressiva (Karras et al., 2018, NVIDIA). O salto de qualidade em 4 anos transformou a área — e criou o problema dos *deepfakes*.

A estrutura da GAN



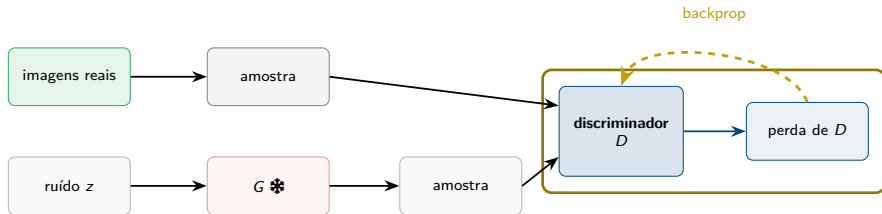
- G transforma ruído em dados; D recebe reais e falsas **misturadas**;
- D quer acertar quem é quem; G quer que D erre;
- duas perdas saem do mesmo D — cada rede usa a sua.

A metáfora clássica: o falsário e o perito



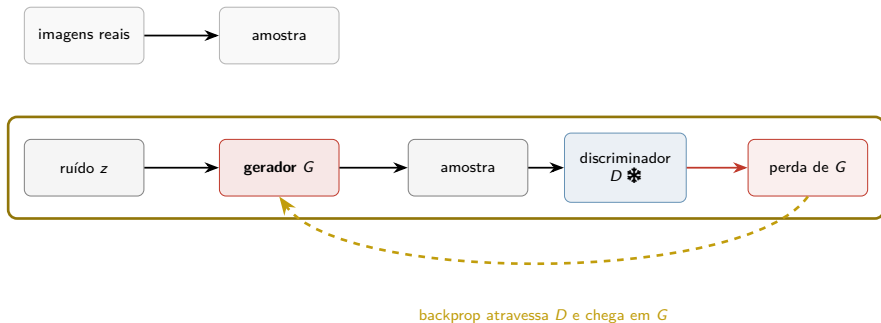
No começo, as “notas” do gerador são toscas e o perito as rejeita com facilidade. A cada rodada o falsário melhora — até que o perito não distinga mais.

Fase 1: treinar o discriminador



- Reais $\rightarrow$ rótulo 1; falsas $\rightarrow$ rótulo 0; perda BCE clássica;
- o gerador fica **congelado** (*): só produz exemplos (no código, `falsas.detach()`);
- só os pesos de D são atualizados.

Fase 2: treinar o gerador



- G quer que D classifique suas falsas como reais (rótulo 1);
- o gradiente **atravessa** D, mas só os pesos de G mudam ($D \nabla$);
- G nunca recebe um pixel real na entrada — só enxerga os dados reais *indiretamente*, pela nota que D (treinado neles) devolve.

Treinamento alternado



- Um passo para D , um passo para G , alternando a cada lote;
- enquanto um treina, o outro fica **constante**;
- começamos por um problema fácil (classificar) para resolver um difícil (gerar);
- se D não consegue separar real de falso *nem no início*, a GAN não treina.

A função minimax: um jogo de dois jogadores

$$\min_G \max_D \underbrace{\mathbb{E}_{x \sim p_{\text{dados}}} [\log D(x)]}_{D \text{ acerta nos reais}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{D \text{ acerta nas falsas}}$$

🔍 O perito D maximiza

- quer $D(x) \rightarrow 1$ (“real é real”);
- quer $D(G(z)) \rightarrow 0$ (“falsa é falsa”).

✂️ O falsário G minimiza

- só toca o 2º termo (o 1º não depende de G);
- quer $D(G(z)) \rightarrow 1$: enganar o perito.

Os dois termos juntos são a **entropia cruzada** entre a distribuição real e a gerada.

Perda minimax modificada: salvando o gradiente

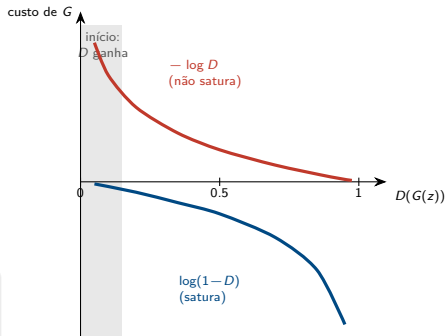
- No **início**, G é ruim e D rejeita as falsas com confiança: $D(G(z)) \approx 0$;
- ali a curva $\log(1 - D(G(z)))$ **varia pouco** (gradiente fraco) frente a $-\log D \Rightarrow G$ aprende devagar;
- conserto do próprio artigo (2014): G passa a **maximizar** $\log D(G(z))$ — íngreme justo onde $D(G(z)) \approx 0$.

No nosso código

$$\mathcal{L}_G = -\mathbb{E}_z [\log D(G(z))]$$

$\iff$ `criterio(D(falsas), uns)`

A BCE com alvo 1 é exatamente $-\log D(G(z))$:
minimizá-la = maximizar $\log D(G(z))$.



No início ($D(G(z)) \approx 0$): a azul $\log(1-D)$ tem gradiente fraco; a vermelha $-\log D$ é íngreme (gradiente forte).

Convergência: um equilíbrio frágil

- À medida que G melhora, D **piora** — as falsas ficam indistinguíveis;
- no equilíbrio ideal, D acerta 50%: **palpite aleatório**;
- mas aí o *feedback* de D vira ruído: se o treino continua, G pode **regredir**.

Diferente de tudo que vimos no curso

A convergência da GAN costuma ser um estado **passageiro**, não estável: não há uma perda única descendo, mas **duas perdas em cabo de guerra**. A curva de G subir não é fracasso — pode ser D melhorando. Na prática, **olhe as amostras**, não só as curvas.

Problemas clássicos de treinamento

Gradientes que somem

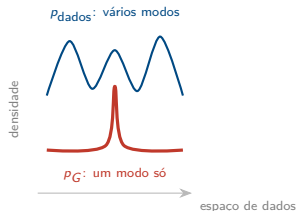
- D bom demais $\rightarrow$ rejeita tudo com confiança $\rightarrow$ gradiente ≈ 0 para G ;
- mitigação: minimax modificada, perda de Wasserstein.

Falha em convergir

- As duas redes oscilam sem chegar a um acordo;
- mitigação: ruído nas entradas de D , penalização de pesos.

Mode collapse

- G encontra *uma* saída convincente e gera só ela (ex.: só o dígito “1”);
- engana D hoje, mas a diversidade morre;
- mitigação: Wasserstein, *unrolled GANs*.



Wasserstein: do perito ao crítico

- A WGAN troca o discriminador por um **crítico**: em vez de “real ou falso”, ele dá uma **nota** (sem **Sigmoid**);
- perdas simples — o crítico quer separar as notas, G quer subir a sua:

$$\underbrace{\mathbb{E}_x[D(x)] - \mathbb{E}_z[D(G(z))]}_{\text{crítico maximiza (separa)}} \quad \underbrace{\mathbb{E}_z[D(G(z))]}_{\text{G maximiza (engana)}}$$

- essa diferença de notas aproxima a **distância de Wasserstein** (“custo de transporte”) entre p_{dados} e p_G , desde que D seja **1-Lipschitz** (daí o *weight clipping* / WGAN-GP);
- gradientes informativos *mesmo* quando as distribuições não se sobrepõem;

Por que importa

Treino mais estável, menos *mode collapse* — a base de GANs modernas (WGAN-GP, StyleGAN usa truques na mesma linha).



GANs na prática e variações

O gerador no MNIST: do ruído à imagem

```
DIM_RUIDO = 100

G = nn.Sequential(
    nn.Linear(DIM_RUIDO, 256), nn.ReLU(),
    nn.Linear(256, 512), nn.ReLU(),
    nn.Linear(512, 784), nn.Tanh(), # saída em [-1, 1]
    nn.Unflatten(1, (1, 28, 28)),
).to(device)
```

- Entrada: $z \in \mathbb{R}^{100}$ sorteado de uma normal;
- a `Tanh` final casa com os dados normalizados em $[-1, 1]$;
- é o **decoder do bloco 1**, mas com o latente vindo do **acaso** em vez de uma imagem real.

O discriminador: um classificador binário comum

```
D = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(784, 512), nn.LeakyReLU(0.2),  
    nn.Linear(512, 256), nn.LeakyReLU(0.2),  
    nn.Linear(256, 1), nn.Sigmoid(), # prob. de ser real  
) .to(device)
```

- Nada de novo: imagem $\rightarrow$ probabilidade;
- **LeakyReLU** evita neurônios mortos — gradiente também na parte negativa, importante porque D é o **canal de feedback** de G .

O loop adversarial completo

```
criterio = nn.BCELoss()
otim_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
otim_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))

for epoca in range(5):
    for reais, _ in loader:
        reais = reais.to(device); b = reais.size(0)
        uns = torch.ones(b, 1, device=device)
        zeros = torch.zeros(b, 1, device=device)

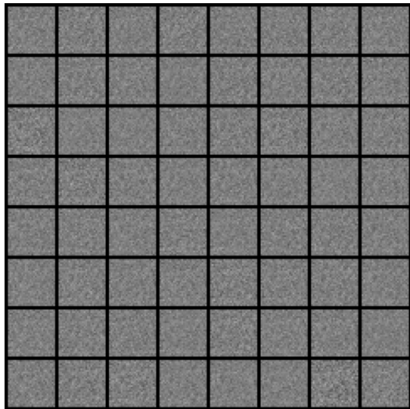
        # fase 1: treina D (G parado)
        falsas = G(torch.randn(b, DIM_RUIDO, device=device))
        perda_D = criterio(D(reais), uns) + criterio(D(falsas.detach()), zeros)
        otim_D.zero_grad(); perda_D.backward(); otim_D.step()

        # fase 2: treina G (D parado)
        perda_G = criterio(D(falsas), uns) # G quer que D diga "real"
        otim_G.zero_grad(); perda_G.backward(); otim_G.step()
```

O `detach()` é a fronteira entre as fases: congela *G* enquanto *D* aprende.

A evolução do gerador

Antes do treino

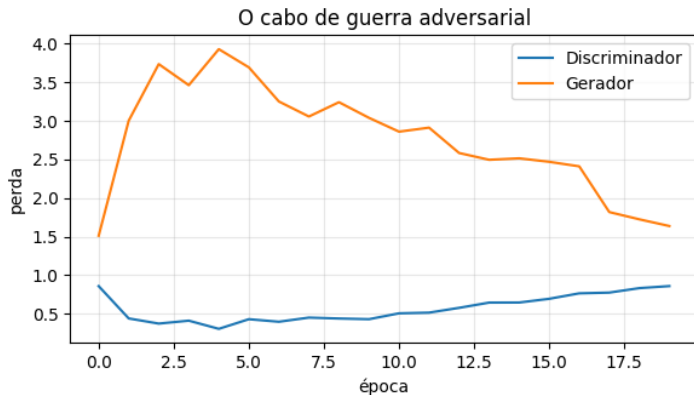


Após 20 épocas



O mesmo `ruido_fixo` antes do treino (esquerda) e após 20 épocas (direita): de estática de TV a dígitos reconhecíveis. Nenhuma dessas imagens existe no MNIST.

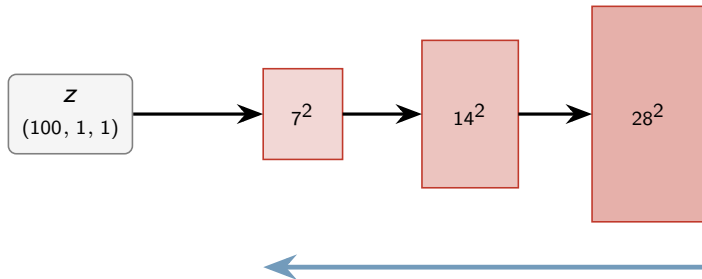
O cabo de guerra nas curvas



As perdas **não** caem juntas como num treino comum: quando uma rede melhora, a perda da outra sobe. O diagnóstico real é **olhar as amostras**.

DCGAN: convoluções no lugar de camadas densas

gerador: $3 \times \text{ConvTranspose2d} (1^2 \rightarrow 7^2 \rightarrow 14^2 \rightarrow 28^2)$



discriminador: o espelho — $\text{Conv2d stride 2} (28^2 \rightarrow 14^2 \rightarrow 7^2 \rightarrow 1)$

- Radford et al. (2015): respeitar a estrutura 2D muda o jogo — o gerador “cresce” a imagem com **convoluções transpostas** (o decoder do bloco 1!);
- **BatchNorm** + **LeakyReLU** estabilizam o treino; o loop adversarial é **idêntico** — só a arquitetura muda.

DCGAN no MNIST

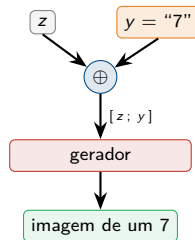
Amostras após a época 10



Os traços saem mais nítidos que na GAN densa: o viés convolucional ajuda o gerador a produzir estrutura local coerente.

GAN condicional: escolhendo o que gerar

- Na GAN básica não controlamos *qual* dígito sai;
- a **cGAN** (Mirza & Osindero, 2014) concatena o **rótulo** y ao ruído z e à entrada de D ;
- o gerador aprende $p(x | y)$: “gere um 7”;
- base de quase todas as aplicações práticas.



pix2pix: tradução de imagem para imagem



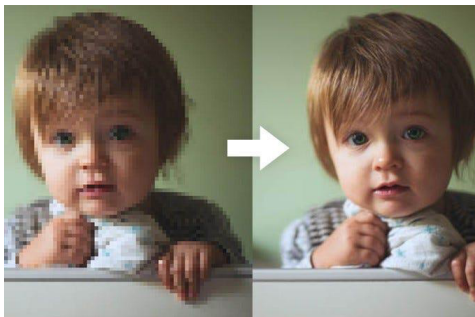
- Isola et al. (2016): a “condição” é uma **imagem inteira** (esboço → foto, mapa → satélite);
- o gerador é um **encoder-decoder** (U-Net!) e o D julga pares (entrada, saída) — exige pares alinhados no treino;
- demo interativa: <https://affinelayer.com/pixsrv/>.

CycleGAN: tradução sem pares



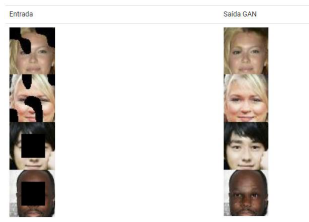
- Zhu et al. (2017): dois geradores ($A \rightarrow B$ e $B \rightarrow A$) e a **perda de ciclo**: cavalo $\rightarrow$ zebra $\rightarrow$ cavalo deve voltar ao original;
- dispensa pares alinhados — bastam dois conjuntos de imagens;
- na versão anterior desta aula, treinamos um CycleGAN para trocar rostos de políticos — a mesma receita serve para qualquer domínio.

Super-resolução



- SRGAN (Ledig et al., 2017): o gerador amplia a imagem **inventando** detalhes plausíveis;
- o discriminador pune resultados “borrados” — por isso GANs superam o MSE puro em nitidez;
- demo: <https://superresolution.streamlit.app/>.

Inpainting e texto para imagem



Inpainting (Yeh et al., 2017):

- preencher blocos ausentes com conteúdo plausível;
- o gerador “imagina” o que falta usando o contexto.

Texto → imagem (Zhang et al., 2016):

- “esta flor tem pétalas amarelas...” → imagem;
- o ancestral direto de DALL-E e Stable Diffusion — que trocaram a GAN por **difusão**, mas mantêm a ideia de gerar do latente.

O outro lado: deepfakes e ética

- As mesmas técnicas geram rostos, vozes e vídeos de pessoas que **nunca existiram** — ou pior, de pessoas que existem;
- fraudes, desinformação e violação de imagem são riscos reais;
- detecção de conteúdo sintético é hoje uma área de pesquisa tão ativa quanto a geração;

Responsabilidade profissional

Quem domina a técnica responde pelo uso: consentimento, transparência (marcar conteúdo sintético) e finalidade legítima não são opcionais.



03_gan.ipynb

- 1 Construir G e D em poucas linhas;
- 2 ver o gerador produzir ruído **antes** do treino;
- 3 escrever o loop adversarial de duas fases;
- 4 acompanhar a evolução com um `ruído_fixo`;
- 5 trocar para o DCGAN reutilizando `ag.treinar_gan`;
- 6 interpolar no espaço z — fechando o ciclo com o bloco 1;
- 7 exercícios: mode collapse e mais épocas.

O mapa da aula



fios comuns: espaço latente ■ redes congeladas como instrumento
■ convolução transposta ■ perdas que medem *percepção*, não pixels

- Autoencoder: aprende representação, mas não amostra bem;
- style transfer: gradiente sobre pixels com VGG congelada;
- GAN: jogo $G \times D$ que aprende a **distribuição** dos dados.

De 2014 para hoje



- GANs dominaram a geração de imagens por quase uma década;
- os **modelos de difusão** (Stable Diffusion, DALL-E, Midjourney) assumiram a frente: o denoiser do bloco 1 limpava em *uma* passagem — a difusão aplica essa ideia **repetidamente**, removendo um pouco de ruído por vez;
- o vocabulário aprendido hoje — latente, gerador, adversarial, condicionamento — é a língua franca de toda a IA generativa.

- Gatys, Ecker, Bethge. *A Neural Algorithm of Artistic Style*, 2015. arxiv.org/abs/1508.06576
- Goodfellow et al. *Generative Adversarial Nets*, NeurIPS 2014.
- Radford, Metz, Chintala. *Unsupervised Representation Learning with DCGANs*, 2015.
- Arjovsky, Chintala, Bottou. *Wasserstein GAN*, 2017.
- Isola et al. *Image-to-Image Translation (pix2pix)*, 2016. ■ Zhu et al. *CycleGAN*, 2017.
- Karras et al. *Progressive Growing of GANs*, ICLR 2018.
- Stevens, Antiga, Viehmann. *Deep Learning with PyTorch* — cap. 2 (modelos pré-treinados, GANs).
- Tutorial oficial PyTorch: *Neural Style Transfer e DCGAN*.

Obrigado! Dúvidas?